Single Liposome Curvature Assay

Image Analysis Protocol

From raw microscopy images to curvature-sorting plots

Version 2.0

For users with no prior coding experience

Overview

This protocol takes you from raw two-channel fluorescence microscopy images to a plot of protein surface density versus liposome radius (curvature).

What the assay measures: Each liposome in your image is a sub-diffraction-limit fluorescent spot. The lipid channel amplitude tells you the liposome's size (proportional to surface area), and the protein channel amplitude tells you how much protein is bound. By calibrating with DLS data, you convert lipid amplitude into a physical radius, then compute protein density = protein_A / ($4\pi R^2$) to see how protein binding depends on curvature.

The four steps:

- **Prepare images** — split microscope TIFFs into channels
- **MATLAB detection** — fit Gaussians to each spot (external code)
- **Analyze MATLAB output** — filter puncta, export amplitudes
- **Plot curvature sorting** — convert to radii, make figures

Quick Primer: Using the Terminal

Every step in this protocol involves typing commands into a **terminal** (also called a command line). Here's how to open one:

macOS: Open Spotlight (Cmd + Space), type `Terminal`, press Enter.

Windows: Press the Windows key, type `PowerShell`, click **Windows PowerShell**.

You'll see a window with a blinking cursor. This is where you type (or paste) all the commands in this protocol. A few things to know:

- **cd** means "change directory." It's how you navigate to a folder. For example, cd Desktop moves you into your Desktop folder.
- **You can paste commands** with Cmd+V (Mac) or Ctrl+V / right-click (Windows). You don't have to type them out.
- **Press Enter** after each command to run it.
- **If something goes wrong**, read the error message — it usually tells you what's missing. The Troubleshooting section at the end covers common errors.

☑ **Tip:** If you see a command that spans multiple lines with \ at the end of each line, that's just one command split across lines for readability. You can paste the whole block at once.

Prerequisites

Before you start, you need these installed on your computer:

Install Python

Check if you already have Python by opening a terminal and typing:

```
python --version
```

If you see something like `Python 3.11.5`, you're good. If you get an error or see Python 2.x, install Python 3:

macOS: Go to `https://www.python.org/downloads/` and download the latest Python 3. Run the installer. Check the box that says "Add Python to PATH."

Windows: Same website, same download. **Important:** during installation, check the box that says "Add python.exe to PATH" — this is on the very first screen of the installer.

ⓘ **Note:** On some Macs, you may need to type python3 instead of python. If python doesn't work, try python3. If python3 works, use python3 everywhere this protocol says python.

Install uv (Package Manager)

uv is a fast Python package manager that replaces pip. It keeps dependencies organized and avoids version conflicts.

macOS/Linux: Open a terminal and run:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Windows PowerShell: Open PowerShell and run:

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

After installing, **close and reopen your terminal** so it picks up the new command. Then verify it works:

```
uv --version
```

You should see a version number like `uv 0.6.x`. If you get "command not found," try closing and reopening the terminal again.

MATLAB + CMEanalysis

You need MATLAB installed with the CMEanalysis code.

If MATLAB is already installed on your device, skip to step 3 below.

1] Download MATLAB – look for Penn instructions. The software should be free for students/faculty.

IMPORTANT – when downloading, you will be prompted to select various toolboxes. These can be installed later, but some will be needed for CMEanalysis. Select the following:

Curve Fitting Toolbox

Image Processing Toolbox

Optimization Toolbox

Statistics Toolbox

Parallel Computing Toolbox

2] Assuming MATLAB is installed, download the needed toolboxes outlined in step 1 if not done already. You can do this by using the Add-On Explorer in the Add-Ons in the Home tab. You will need to restart MATLAB for each toolbox installed.

3] Install CMEAnalysis code somewhere on your device. Go to this link, and click the green "Code button", then download the zip. Unzip the folder on your computer, and save it to a good location.

DLS Data

You need the following from your Malvern Zetasizer DLS measurement: the Z-average diameter (nm) and PDI from the cumulants summary report, and the intensity-weighted size distribution exported as an Excel spreadsheet. See the DLS Calibration section below for details on how to prepare this spreadsheet.

Download the Code and Set Up

1. **Clone or download the repo from GitHub.** Go to https://github.com/slele225/liposome-curvature-assay. Option A: if you have git, open Terminal and run: git clone https://github.com/slele225/liposome-curvature-assay.git (then skip to step 3). Option B: click the green Code button, then Download ZIP.

2. **Unzip it.** Double-click the .zip file to extract it. You should now have a folder called liposome-curvature-assay.

3. **Move it somewhere you'll remember.** We recommend putting it on your Desktop or in your Documents folder. For this protocol, we'll assume it's on your Desktop.

4. **Open a terminal and navigate to the folder.**

macOS:

```
cd ~/Desktop/liposome-curvature-assay
```

Windows PowerShell:

```
cd $HOME\Desktop\liposome-curvature-assay
```

✅ **Tip:** You can also type cd followed by a space, then drag and drop the folder from Finder/File Explorer into the terminal window. It will auto-fill the path for you.

5. **Create a virtual environment.** This makes a private space for the Python packages so they don't interfere with anything else on your computer.

```
uv venv
```

You should see a message like `Creating virtual environment at: .venv`. A new hidden folder called .venv now exists inside the repo.

6. **Activate the virtual environment.** This tells your terminal to use the packages in .venv.

macOS/Linux:

```
source .venv/bin/activate
```

Windows PowerShell:

```
.venv\Scripts\Activate
```

How to tell it worked: Your terminal prompt should now start with `(.venv)`. For example:

```
(.venv) yourname@computer ~ %
```

⚠ **You need to activate the venv EVERY TIME you open a new terminal window. If you close the terminal and come back later, repeat this cd and activate step.**

7. **Install the Python packages.**

```
uv pip install -r requirements.txt
```

This downloads and installs numpy, matplotlib, tifffile, h5py, openpyxl, pandas, and scipy. It takes about a minute.

8. **Verify everything works.**

```
python plot_curvature.py --help
```

You should see a help message listing all the options. If you get an error, check that (.venv) appears in your terminal prompt.

Folder Structure: Where to Put Your Files

The repo has three folders for organizing your work. **All your data and outputs go inside the repo folder.** Here is the structure:

```
liposome-curvature-assay/
├── data/                 ← YOUR DATA GOES HERE
│   ├── dls_data.xlsx     ← DLS Excel export
│   └── experiment_name/  ← one subfolder per experiment
│       └── 488nm_.../    ← (created by Step 1)
│           ├── cell1/
│           │   ├── ch1/
│           │   └── ch2/
│           └── filtered_puncta_A_values.txt
```

```
├── figures/              ← PLOTS SAVED HERE
├── test_data/            ← EXAMPLE DATA(TIFF IMAGES AND DLS)
├── prepare_input.py
├── analyze_matlab.py
├── dls_calibration.py
├── plot_curvature.py
├── plot_dls.py
├── plot_dls_comparison.py
├── plot_histograms.py
├── plot_overlay.py
├── requirements.txt
└── README.md
```

data/ — Put your raw TIFF images in a subfolder here (e.g., data/march_3_experiment/). The prepare_input.py script will create the MATLAB-ready folder structure inside data/. Also put your DLS .xlsx file here.

figures/ — This is where your final plots get saved. The plot_curvature.py script creates this folder automatically when you use --save-dir figures/.

ⓘ **Note:** The data/ and figures/ folders are gitignored, meaning they won't be uploaded if you use git. Your data stays on your computer only.

Step 1: Prepare Images

What this does: Takes your raw multi-frame TIFF files from the microscope, splits them into separate channels, and organizes them into the folder structure that MATLAB expects. Files are automatically grouped by detector voltage from the Olympus FV3000 metadata.

Before you start:

- Put all your raw .tif files from one experiment into a subfolder inside data/. For example: data/march_3_experiment/
- Know which frame index corresponds to which channel. If unsure, open a TIFF in ImageJ and check which frame is lipid vs. protein.

In your terminal (make sure you're in the repo folder and venv is activated):

1. **Make sure you're in the right place:**

```
cd ~/Desktop/liposome-curvature-assay
```

(Or wherever you put the repo. Skip this if you're already there.)

2. **Activate the venv if you haven't already:**

macOS: `source .venv/bin/activate`

Windows: `.venv\Scripts\Activate`

3. **Run the command** (edit the paths and frame numbers for your data):

macOS/Linux:

```
python prepare_input.py \
    --input  data/march_3_experiment \
    --output data/march_3_experiment_matlab \
    --frames 2,0 \
    --crop 1
```

Windows PowerShell (use backtick ` instead of backslash \):

```
python prepare_input.py `
    --input  data\march_3_experiment `
    --output data\march_3_experiment_matlab `
    --frames 2,0 `
    --crop 1
```

What the arguments mean:

Argument	What it means
`--input`	Folder with your raw .tif files, inside data/
`--output`	Where to save the split channels, also inside data/
`--frames`	Comma-separated frame order. "2,0" means frame 2 → ch1 (lipid), frame 0 → ch2 (protein). Check your microscope settings.
`--crop`	Center crop divisor. 1 = no crop. 2 = center quarter crop. Use if your image has dark borders.

4. **Check the output.** The script prints a summary. You should see a new folder inside data/ with subfolders for each voltage group, cell, and channel.

Expected output structure:

```
data/march_3_experiment_matlab/
  └── 488nm_530V_561nm_500V/
      ├── cell1/
      │   ├── ch1/   ← lipid
      │   └── ch2/   ← protein
      └── cell2/
          ├── ch1/
          └── ch2/
```

Step 2: Run MATLAB Detection

What this does: Fits 2D Gaussians to every sub-diffraction-limit fluorescent spot in each channel image. This identifies individual liposomes and measures their amplitudes.

These instructions use CMEAnalysis, which is a separate code base that should have been installed from the prerequisites section above.

1. **Open MATLAB and navigate to the CMEAnalysis folder.** In the MATLAB command window, type the following two commands. Replace the path with the actual location where you saved CMEAnalysis in the prerequisites step:

```
cd('/path/to/cmeAnalysis-master/software')
addpath(genpath(pwd))
```

2. **Load the image folder.** Run the following command in the MATLAB command window:

```
data = loadConditionData();
```

This will open a selection window asking for a condition folder. Select the folder produced in the previous step titled with the condition (voltages).

3. **Select channels.** The program will ask you to select the "first (master) channel." Select the lipid channel folder (should be ch1). It will then ask you to select all the remaining folders. Select the ch2 folder.

4. **Enter fluorophore names.** The program will ask for a fluorescent marker for each channel. Input the name of the fluorophore corresponding to each channel. Use `texasred` for Texas Red dye, and `egfp` for EGFP. If it does not recognize the fluorophore name, it will ask you to input the emission wavelength in nm.

5. **Run the detection.** This detection mode determines the Gaussian-approximated PSF standard deviation from the images themselves (the default). Type:

```
runDetection(data);
```

If you have already run a detection and want to redo it:

```
runDetection(data, 'Overwrite', true);
```

If you know what sigma to use from separate experiments or calculations:

```
runDetection(data, 'Overwrite', true, 'Sigma', sigma_value);
```

Replace `sigma_value` with your number. Note that sigma is the standard deviation, not the variance.

6. **Wait for completion.** The code will show its progress and take a few minutes at most. It will report the detected standard deviations for each channel across all images and show graphs with this data — you can close them.

When it's done, verify that the master channel (ch1) now has a Detection subfolder. Note that only the master/lipid channel gets the Detection folder — ch2 will just have the TIFF file:

```
data/march_3_experiment_matlab/488nm_530V_561nm_500V/
  └── cell1/
      ├── ch1/
      │   └── Detection/
      │       └── detection_v2.mat   ← this file must exist
      └── ch2/
          └── (TIFF image only, no Detection folder)
```

ⓘ **Note:** The .mat files contain a struct called frameInfo with fields A (amplitude), c (background), and hval_Ar (hypothesis test value). The next step reads these.

Step 3: Analyze MATLAB Output

What this does: Reads the .mat detection files, filters out noise and poorly-fit spots, and exports a text file with the amplitude values for every liposome that passed quality control.

In your terminal (repo folder, venv activated):

macOS/Linux:

```
python analyze_matlab.py \
    --input data/march_3_experiment_matlab/488nm_530V_561nm_500V \
    --channels ch1,ch2 \
    --lipid-channel ch1 \
    --k-std 2.0 \
    --output-name filtered_puncta_A_values.txt
```

Windows PowerShell:

```
python analyze_matlab.py `
    --input data\march_3_experiment_matlab\488nm_530V_561nm_500V `
    --channels ch1,ch2 `
    --lipid-channel ch1 `
    --k-std 2.0 `
    --output-name filtered_puncta_A_values.txt
```

What the arguments mean:

Argument	What it means
`--input`	The voltage group folder inside data/ (from Step 1 output)
`--channels`	Channel names in order, matching the folder names: ch1,ch2
`--lipid-channel`	Which channel is the lipid dye (used for thresholding)

`--k-std`	How strict the filter is. Default 2.0 means: keep spots where amplitude > mean(background) + 2×std(background). Lower = keep more points. Higher = stricter.
`--output-name`	Name of the output file. Gets saved inside the --input folder.

What to expect: The script prints a per-cell summary showing how many puncta were kept vs. seen. At the end, it saves a tab-separated text file:

```
data/march_3_experiment_matlab/488nm_530V_561nm_500V/filtered_puncta_A_values.txt
```

This file has columns: `source_image`, `A_ch1` (lipid amplitude), `A_ch2` (protein amplitude). Each row is one liposome.

Step 4: Plot Curvature Sorting

What this does: Converts lipid amplitudes into physical liposome radii using your DLS mean diameter, computes protein surface density for each liposome, and generates a scatter plot.

You need: Either a conversion factor from dls_calibration.py (recommended) or the mean liposome diameter in nm from your DLS measurement. Run dls_calibration.py first to get both options.

In your terminal (repo folder, venv activated):

macOS/Linux:

```
python plot_curvature.py \
    --input
data/march_3_experiment_matlab/488nm_530V_561nm_500V/filtered_puncta_A_values.txt \
    --conversion-factor 1.234 \
    --lipid-col A_ch1 \
    --protein-col A_ch2 \
    --bin-width 0.5 \
    --save-dir figures/
```

Windows PowerShell:

```
python plot_curvature.py `
    --input
data\march_3_experiment_matlab\488nm_530V_561nm_500V\filtered_puncta_A_values.txt `
    --conversion-factor 1.234 `
    --lipid-col A_ch1 `
    --protein-col A_ch2 `
    --bin-width 0.5 `
    --save-dir figures/
```

What the arguments mean:

Argument	What it means
`--input`	Path to the filtered_puncta text file from Step 3
`--conversion-factor`	Conversion factor from dls_calibration.py (maps sqrt(A) to diameter in nm). Alternatively, use --dls-mean-diameter with the mean diameter in nm. Provide one or the other.
`--lipid-col`	Column name for lipid amplitude (default: A_ch1)
`--protein-col`	Column name for protein amplitude (default: A_ch2)
`--bin-width`	Width of radius bins for the averaged curve. Smaller = noisier, larger = smoother. Default 0.5 nm.
`--save-dir`	Where to save the figure. Use figures/ to keep things organized inside the repo.

Output: A PNG file saved in the figures/ folder:

```
figures/488nm_530V_561nm_500V__protein_density_vs_radius.png
```

How the Size Conversion Works

The lipid fluorescence amplitude is proportional to the amount of dye, which is proportional to surface area. So:

- sqrt(amplitude) is proportional to diameter
- scale factor = DLS_mean_diameter / mean(sqrt(all lipid amplitudes))
- Each liposome's radius = sqrt(its lipid_A) × scale / 2
- Protein density = protein_A / ($4\pi R^2$)

ⓘ **Note:** Two calibration methods are available. The recommended method uses log-normal fitting (run dls_calibration.py with your DLS spreadsheet and filtered puncta file) which outputs a conversion factor for use with --conversion-factor. The simpler ratio-of-means method uses --dls-mean-diameter. Both convert sqrt(lipid amplitude) to physical diameter in nm.

Interpreting the Plot

- **X-axis** = liposome radius (nm). Smaller radius = higher curvature.
- **Y-axis** = protein surface density (amplitude per nm^2). Higher = more protein per unit area.
- **Transparent dots** = individual liposomes.
- **Solid dots** = binned average (the trend line you care about).

What the trend tells you:

- **Binned average goes UP toward smaller radii** → your protein preferentially binds high-curvature membranes (curvature sensing).
- **Flat line** → protein binds independent of curvature.
- **Binned average goes DOWN toward smaller radii** → protein avoids high curvature.

Optional: Plot Histograms

You can visualize the amplitude distributions and estimated liposome sizes using the histogram script. This is useful for sanity-checking your data before making the curvature-sorting plot, or for lipid-only experiments where you don't have a protein channel.

With both lipid and protein channels (the typical case):

macOS/Linux:

```
python plot_histograms.py \
    --input
data/march_3_experiment_matlab/488nm_530V_561nm_500V/filtered_puncta_A_values.txt \
    --lipid-col A_ch1 \
    --protein-col A_ch2 \
    --conversion-factor 1.234 \
    --save-dir figures/
```

Windows PowerShell:

```
python plot_histograms.py `
    --input
data\march_3_experiment_matlab\488nm_530V_561nm_500V\filtered_puncta_A_values.txt `
    --lipid-col A_ch1 `
    --protein-col A_ch2 `
    --conversion-factor 1.234 `
    --save-dir figures/
```

Lipid-only (no protein channel — just omit --protein-col):

```
python plot_histograms.py \
    --input data/.../filtered_puncta_A_values.txt \
    --lipid-col A_ch1 \
    --conversion-factor 1.234 \
    --save-dir figures/
```

Windows PowerShell:

```
python plot_histograms.py `
    --input data\...\filtered_puncta_A_values.txt `
    --lipid-col A_ch1 `
    --conversion-factor 1.234 `
    --save-dir figures/
```

To view log-transformed amplitudes (useful for visualizing spread), add `--transform log_sqrt` to any of the above commands. Options are `raw` (default), `sqrt`, or `log_sqrt`.

What it produces: Amplitude histograms for each channel (with mean/median lines), a protein amplitude histogram (if protein column provided), and an estimated diameter distribution calibrated against your DLS data (if `--conversion-factor` is provided). The diameter distribution is especially useful — if it looks nothing like your DLS, something is off with the calibration or filtering.

Optional: DLS Calibration Script

The script fits log-normal distributions to both the DLS intensity distribution and the fluorescence sqrt(A) distribution simultaneously. It outputs conversion factors that map sqrt(amplitude) to physical diameter in nm. It also computes a simple ratio-of-means conversion for comparison. It uses the **Intensity** distribution. The ratio-of-means conversion uses the **number** distribution. See the example data to see how the DLS data should be formatted in the spreadsheet.

macOS/Linux:

```
python dls_calibration.py --dls-input data/dls_data.xlsx --fluor-input
data/.../filtered_puncta_A_values.txt --z-avg 131.0 --pdi 0.08 --save-dir figures/
```

Windows PowerShell:

```
python dls_calibration.py --dls-input data\dls_data.xlsx --fluor-input
data\...\filtered_puncta_A_values.txt --z-avg 131.0 --pdi 0.08 --save-dir figures/
```

The `-z-avg` and `--pdi` commands are optional. Without them, only the ratio-of-means is computed.

It will print three conversion factors and show how to use each. Use the log-normal conversion factor with --conversion-factor, or the ratio-of-means diameter with --dls-mean-diameter, in Steps 4a and 4b.

Note: The DLS spreadsheet must include the intensity distribution (for log-normal fitting) and the number distribution (for ratio-of-means). Export the size distribution from the Zetasizer software and copy it into an Excel file with the standard section headers (“X Intensity”, “X Volume”, “X Number”). You also need the Z-average and PDI from the Zetasizer cumulants summary report.

Optional: Plotting the DLS and Comparing DLS and Fluorescence Distributions

These utility scripts let you visualize the DLS size distribution and compare it side-by-side with your fluorescence data. Use them as a sanity check to make sure your detected liposome population matches what DLS measured.

Plot DLS Distribution Only

Plots the DLS distribution from your Zetasizer spreadsheet, zoomed to show only the bins with signal. You can choose the number or intensity distribution.

macOS/Linux:

```
python plot_dls.py data/dls_data.xlsx
```

Windows PowerShell:

```
python plot_dls.py data\dls_data.xlsx
```

Add `--log` to plot log(diameter) instead. Add `--distribution intensity` to use the intensity distribution instead of number. Add `--zoom-pct 95` to zoom to 95% of the data (default is 95%).

Compare DLS and Fluorescence Distributions

Plots the DLS distribution side-by-side with the sqrt(A) histograms for each fluorescence channel. Use this to check whether the shapes match — if they do, your detection is picking up the correct liposome population. You specify channels as pairs: the channel index (0 for ch1, 1 for ch2, etc.) followed by a label.

macOS/Linux:

```
python plot_dls_comparison.py \
    --dls-input data/dls_data.xlsx \
    --fluor-input data/.../filtered_puncta_A_values.txt \
    --channels 0 Lipid 1 EGFP \
    --zoom-pct 95 --bins 200 --save-dir figures/
```

Windows PowerShell:

```
python plot_dls_comparison.py `
    --dls-input data\dls_data.xlsx `
    --fluor-input data\...\filtered_puncta_A_values.txt `
    --channels 0 Lipid 1 EGFP `
    --zoom-pct 95 --bins 200 --save-dir figures/
```

How channels work: You specify channels as pairs of index and label. Index 0 corresponds to A_ch1 (typically lipid), index 1 to A_ch2 (typically protein), and so on. The label is what appears in the plot title (e.g., “Lipid”, “EGFP”). You can include as many channels as you want — each gets its own panel.

Zoom: The `--zoom-pct` flag controls how much of the data to show. Default is 100% (no zoom). Set to 95 to zoom into the central 95% and hide outlier tails. The DLS and fluorescence panels are each zoomed independently based on their own distributions.

Optional: Normalized Overlay Across Experiments

If you have multiple experiments (different proteins, conditions, or replicates) and want to compare their curvature sorting on a single plot, use the overlay script. It normalizes each curve so the largest-radius bin equals 1, then overlays just the binned averages. The y-axis shows fold-enrichment at high curvature relative to flat membranes.

Key detail: Each input file gets its own conversion factor (from dls_calibration.py), specified with a colon after the filepath. This is important because different liposome stocks may have different size distributions.

macOS/Linux:

```
python plot_overlay.py \
    --input data/cond1/filtered_puncta_A_values.txt:1.234 \
           data/cond2/filtered_puncta_A_values.txt:1.567 \
    --labels "WT protein" "Mutant K58A" \
    --save-dir figures/
```

Windows PowerShell:

```
python plot_overlay.py `
    --input data\cond1\filtered_puncta_A_values.txt:1.234 `
           data\cond2\filtered_puncta_A_values.txt:1.567 `
    --labels "WT protein" "Mutant K58A" `
    --save-dir figures/
```

The format is: `path/to/file.txt:conversion_factor` — note the colon between the filepath and the conversion factor. Each file gets its own conversion factor from dls_calibration.py because different experiments may use different liposome stocks.

Labels: The `--labels` flag lets you give each curve a readable name for the plot legend (e.g., the protein name or condition). If you leave it off, the script uses the parent folder name of each input file.

Output: A PNG saved in figures/ with all curves overlaid. A dashed gray line at y = 1 marks the baseline. Curves that rise above 1 toward smaller radii indicate curvature sensing.

Quick Reference: Every-Time Checklist

After the initial setup is done, here's what you do each time you have new data:

1. Open a terminal
2. Navigate to the repo: **cd ~/Desktop/liposome-curvature-assay**

3. Activate the venv: **source .venv/bin/activate** (Mac) or **.venv\\Scripts\\Activate** (Windows)
4. Copy your raw TIFFs into a new subfolder inside **data/**
5. Run **prepare_input.py** (Step 1)
6. Run MATLAB detection (Step 2)
7. Run **analyze_matlab.py** (Step 3)
8. Run **plot_curvature.py** with your conversion factor (or DLS mean diameter) and **--save-dir figures/** (Step 4)
9. Find your plot in the **figures/** folder

Troubleshooting

Problem	Solution
"command not found: python"	Try python3 instead. On some Macs, python isn't linked but python3 is. If neither works, Python isn't installed — see Prerequisites.
"command not found: uv"	Close and reopen your terminal after installing uv. If still not found, reinstall uv using the commands in Prerequisites.
"No module named tifffile" or similar	Your virtual environment isn't activated. Run: source .venv/bin/activate (Mac) or .venv\Scripts\Activate (Windows). Look for (.venv) in your prompt.
"No .tif files found"	Check that --input points to the folder containing your TIFFs, not a parent folder or a subfolder.
"Frames out of range"	Your --frames indices don't match the TIFF. Open one in ImageJ to check how many frames it has. Frame indices start at 0.
"missing detection file"	MATLAB detection hasn't been run on that cell/channel yet. Go back to Step 2.
"No puncta passed the filters"	Threshold too strict or very low signal. Try --k-std 1.5 or --k-std 1.0.
Very noisy plot	Increase --bin-width (try 1.0 or 2.0). Or double-check your DLS mean diameter is correct.
"Permission denied" on macOS	For install.sh: run chmod +x scripts/install.sh first. For .venv: delete the .venv folder and recreate with uv venv.
I closed my terminal and now nothing works	You need to cd back to the repo folder and activate the venv again. See the Quick Reference checklist.